COMP 3920 - Database Systems 1

Introduction to Transactions

Introduction:
The goal of this document is to serve as a brief introduction to database transactions.

Transactions are groups of commands, typically DML commands such as INSERTs, UPDATEs and DELETEs. The changes resulting from the commands in a group are stored *temporarily* and can be *aborted* (or ROLLBACKed) if an undesired effect is detected.

Why would you want to store changes from an INSERT, UPDATE or DELETE temporarily?

- What if you forgot the WHERE clause in your UPDATE or DELETE?
- What if you do an UPDATE or DELETE and thought you were only modifying a small number of rows, but more rows than you expected got modified?
- What if you make a small typo in your INSERT, UPDATE or DELETE and you don't get the results you wanted?
- What if you accidentally swapped the `first_name` and `last_name` when INSERTing a person?
- All or nothing. We might have to do a number of UPDATEs, INSERTs and/or DELETEs to keep the database consistent. For example, if we sell a product, we might want to do all of these things at once.
  1. Create Invoice.
  2. Decrease product inventory.
  3. Decrease customer balance.

  What if we can't bill the customer, because their balance is too low? Should we still create the invoice and decrease inventory? No, probably not. It's all or nothing after all.

In all of the scenarios above it would be nice to have a way to examine the result of DML commands before the changes take effect. This is one advantage of transactions.

The basic transaction looks like this:

```
BEGIN TRANSACTION;

<DML Statement #1>

<DML Statement #2>

…

ROLLBACK;
OR
COMMIT;
```

To start a transaction type: `BEGIN TRANSACTION;` and execute it.
MySQL will respond with: `0 rows affected.`

Now you are free to make any DML changes to the database you which and your c*hanges will only be visible to you*.

A Transaction ends with either 1 of 2 things happen. Either you are happy with the changes and want to keep them. Or you find something wrong with the changes and wish to abort them.

If you wish to ***keep*** the changes type and execute this command: `COMMIT;`

If you wish to ***abort*** the changes type and execute this command: `ROLLBACK;`

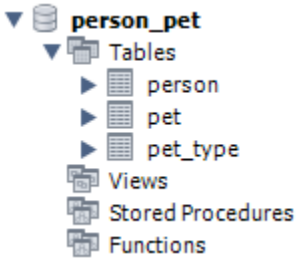
Remember: Other people logged in to the database cannot see your changes unless you `COMMIT` your changes. If you `ROLLBACK` your changes, it's like they never happened.

Step 1:
Open MySQL Workbench.

Step 2:
Create the `Person_Pet` database (if you don't already have it).

Open `person_pet_backup.sql` and execute **all** the commands in the file.

This should create a database called `person_pet` with the following tables (and some data inside the tables):



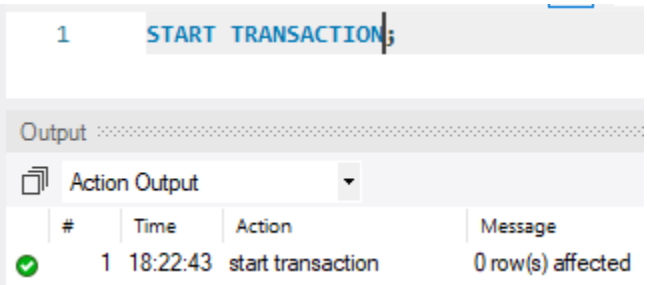
Step 3:
Practice aborting a transaction with ROLLBACK.

In a new Query Window (using the `Person_Pet` database).

Create a new Transaction -

Type and Execute the following:

```
START TRANSACTION;
```



Let's ***accidentally*** delete all the pets in the `pet` table.

What we wanted to do was delete only Thumper, but, if we forgot to put a `WHERE` clause on our `DELETE` command we could DELETE a lot more than we wanted!

Type and execute:

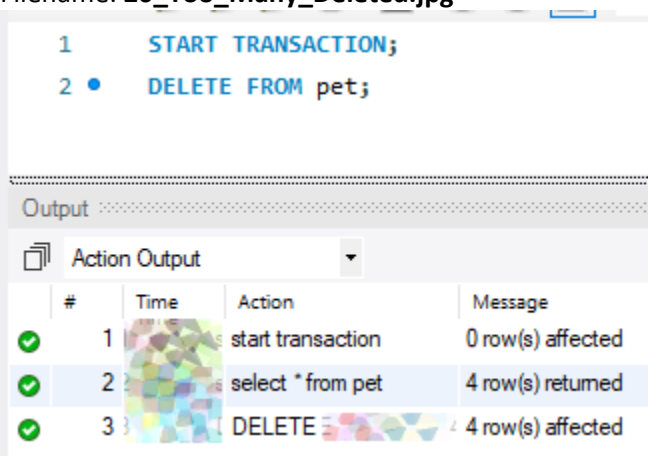
```
DELETE FROM Pet;
```

Uh, Oh! We wanted to see only **1 row affected**, but we got **way more**!

Submit a screenshot of your query `DELETE FROM pet;` **<u>and</u>** the number of rows affected.

Filename: **20_Too_Many_Deleted.jpg**

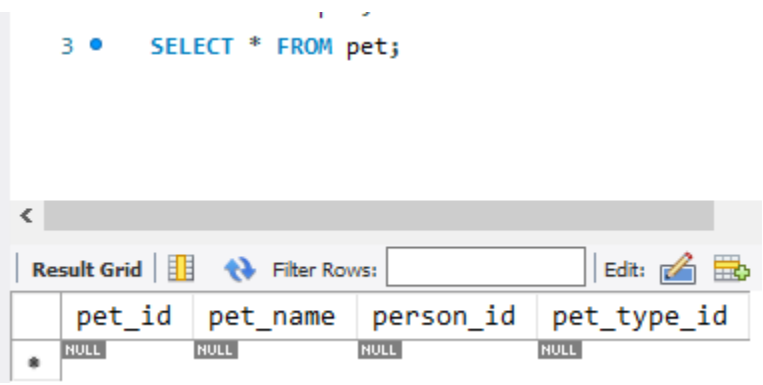
Submit



We can confirm that all the pets are deleted (at least temporarily) by doing a SELECT statement.

Type and execute:

```
SELECT * FROM Pet;
```



Yup. All gone! ☹️ Good thing we used a transaction!

We can ROLLBACK our changes by using the `ROLLBACK` command -

Type and execute:

```
ROLLBACK;
```

```
4 •  ROLLBACK;
```

✔ 6	18:29:38	ROLLBACK	0 row(s) affected

All the pets should be back -

Type and execute:

```
SELECT * FROM pet;
```

Submit a screenshot of your query of the `pet` table **<u>and</u>** the results.

Filename: **21_All_Pets_are_Back.jpg**

Submit

```
3 •  SELECT * FROM pet;
4 •  ROLLBACK;
```

Result Grid | Filter Rows: | Edit:

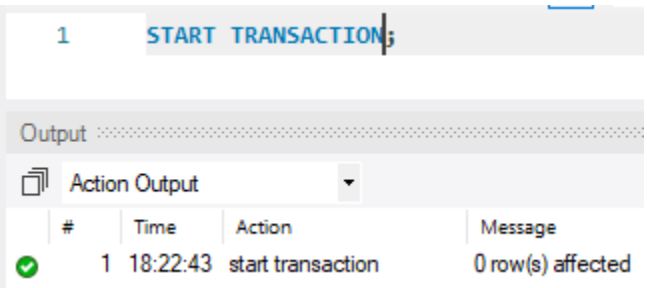
pet_id	pet_name	person_id	pet_type_id
[illegible]	Fluffers	[illegible]	
	Rex		
	Bubbles		
	Thumper		
	NULL		

Step 4:
Practice keeping the changes from a transaction with `COMMIT`.

Create a new Transaction -

Type and Execute the following:

```
START TRANSACTION;
```



Let's add a new pet - Fido (dog) to George Thornton.

Write an `INSERT` statement to insert George Thornton to the `Person` table.

Write a `SELECT` statement to determine the `person_id` for the new row created for George.

Write an `INSERT` statement to INSERT Fido with `pet_type_id` of type **dog** and George as the owner.

To confirm that this is what we want, write a simple `SELECT` statement to list all pets and their owners (don't list pets without owners).

If you are happy with the results, let's `COMMIT` our transaction -

Type and Execute the following:

```
COMMIT;
```

Submit a screenshot of your `COMMIT` statement **and** the results.

Filename: **22_Ready_for_Commitment.jpg**

Submit

```
15 •  COMMIT;
```

Output

Action Output

#	Time	Action	Message
14		COMMIT	0 row(s) affected

Re-run your simple `SELECT` statement to list all pets and their owners to confirm the COMMIT took effect.

Submit a screenshot of your simple `SELECT` statement **and** the results.

Filename: **23_Fully_Committed.jpg**

Submit

```
10 •  select
11    from
12    join
13    join
14
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	pet_name	pet_type
George	Thornton	Fido	dog

Awesome! All Done!